



Volume 1 - Architecture et exécution

Comment le programme passe de la ligne de commande aux sockets, threads, codecs et exports

Ce volume décrit le trajet global d'une trame et les responsabilités de chaque couche. Il sert de carte de navigation avant les volumes consacrés aux commandes détaillées.

Table des matières

1	Architecture en une phrase	2
2	Frontière processus : les deux main	2
3	Protocole STGS v1	2
3.1	Layout binaire	2
3.2	CRC : pourquoi 0xEDB88320	3
4	Pourquoi TCP et UDP ne suivent pas le même chemin	3
4.1	UDP	3
4.2	TCP	3
5	Pipeline concurrent et déterminisme	3
5.1	Backpressure	4
5.2	Pourquoi réordonner ?	4
6	Couche applicative STGA	4
7	Arrêt Ctrl+C sans handler C++ dangereux	4
8	Affichage terminal	4
9	Garanties et non-garanties	5

1 Architecture en une phrase

STGS sépare volontairement cinq responsabilités : **entrée** (TCP/UDP/replay), **framing et validation** (STGS v1 + CRC), **pipeline concurrent borné**, **interprétation applicative** (STGA texte/signal) et **sortie** (terminal + CSV/JSON).

Flux global d'une trame réseau

```
socket TCP/UDP → NetworkServer → GroundStationPipeline::submit() → rawQueue →
decoderLoop() → decodeFrame() → decodedQueue → writerLoop() / réordonnancement
→ StationHealthMonitor → decodeApplicationPayload() → TerminalUi →
TelemetryOutput
```

2 Frontière processus : les deux main

Les deux exécutable applicatifs gardent un `main()` volontairement minimal. Leur rôle est de :

1. demander au parseur CLI de construire une configuration validée ;
2. appeler la fonction `run()` de l'application ;
3. convertir toute exception non gérée en message `stderr` et code retour 1.

Source : `apps/ground_station.cpp`, lignes 15–27

Source : `apps/satellite_simulator.cpp`, lignes 14–26

Pourquoi ce choix ?

Un `main()` court rend la frontière processus évidente et testable mentalement. La logique métier n'est pas mélangée avec le traitement de `argc/argv` ou les codes de sortie.

3 Protocole STGS v1

3.1 Layout binaire

Tous les entiers sont sérialisés en big-endian. La taille du header est dérivée des tailles wire et verrouillée par `static_assert`.

Champ	Taille	Rôle
MAGIC	4 octets	Valeur ASCII STGS , resynchronisation TCP.
VERSION	1	Version du protocole, actuellement 1.
SATELLITE_ID	2	Identifiant logique de la source.
TIMESTAMP_MS	8	Horodatage en millisecondes.
TEMPERATURE_C	4	Float32, doit être fini.
BATTERY_PERCENT	1	Entier 0..100.
STATUS	1	NOMINAL/WARNING/CRITICAL/SAFE_MODE.
PAYLOAD_LEN	2	Taille du payload.
PAYLOAD	0..4096	Données opaques ou enveloppe STGA.
CRC32	4	CRC-32/ISO-HDLC sur tout ce qui précède.

Source : `include/stgs/TelemetryFrame.hpp`, lignes 22–71

Limites wire

Header : 23 octets. CRC : 4 octets. Payload : 4096 octets maximum. Trame minimale : 27 octets.
Trame maximale : $23 + 4096 + 4 = 4123$ octets.

3.2 CRC : pourquoi 0xEDB88320

Le CRC est exactement CRC-32/ISO-HDLC : width=32, poly canonique 0x04C11DB7, init 0xFFFFFFFF, refin/refout=true, xorout 0xFFFFFFFF. L'implémentation parcourt les bits LSB-first ; elle utilise donc le polynôme réfléchi 0xEDB88320. Le vecteur standard 123456789 doit produire 0xCBF43926.

Calcul du CRC

```
crc32(bytes) → crc = 0xFFFFFFFF → index = (crc XOR byte) & 0xFF → crc = (crc » 8) XOR table[index] → crc XOR 0xFFFFFFFF
```

Source : include/stgs/Crc32.hpp, lignes 18–42 et **Source :** src/Crc32.cpp, lignes 42–54

4 Pourquoi TCP et UDP ne suivent pas le même chemin

4.1 UDP

UDP conserve les frontières de datagrammes. Le serveur alloue un buffer de `MaxFrameSize`, utilise `recvfrom(..., MSG_TRUNC, ...)` et rejette un datagramme plus grand que la taille maximale au lieu de transmettre au codec un préfixe tronqué.

Source : src/NetworkServerUdp.cpp, lignes 18–77

4.2 TCP

TCP est un flux d'octets : une lecture peut contenir une demi-trame, une trame, plusieurs trames, ou du bruit avant le magic. Chaque client possède donc son propre `StreamFrameExtractor`.

Framing TCP

```
recv() → StreamFrameExtractor::feed(fragment) → recherche magic STGS → attente du header complet → lecture PAYLOAD_LEN → attente de header + payload + CRC → émission d'un candidat complet
```

L'extracteur ne valide pas encore le CRC : il ne fait que retrouver les frontières. `decodeFrame()` prend ensuite la décision de validation/rejet.

Source : include/stgs/FrameCodec.hpp, lignes 67–92 et **Source :** src/FrameCodec.cpp, lignes 221–263

5 Pipeline concurrent et déterminisme

La station crée trois catégories de threads :

- un thread de progression terminal ;
- un writer unique ;
- N décodeurs configurés par `-decoder-threads`.

Démarrage et arrêt des workers

```
GroundStationPipeline::run() → startWorkers() → receiveInput() → rawQueue.close() → join décodeurs → decodedQueue.close() → join writer → stop progress thread → rethrow erreur asynchrone éventuelle
```

Source : apps/ground_station/GroundStationPipeline.cpp, lignes 49–128

5.1 Backpressure

Les files de production sont bornées. Lorsque la file est pleine, `push()` attend qu'un consommateur libère une place. La surcharge se traduit donc par ralentissement du producteur, pas par croissance mémoire illimitée.

Source : `include/stgs/BlockingQueue.hpp`, lignes 22–113

5.2 Pourquoi réordonner ?

Avec plusieurs workers, le résultat de la trame 12 peut terminer avant la trame 11. Chaque trame reçoit donc une séquence monotone avant décodage. Le writer conserve les résultats dans une `std::map` et ne consomme que `expectedSequence`. Ainsi la santé et le CSV/JSON sont indépendants du nombre de workers.

Source : `apps/ground_station/GroundStationWorkers.cpp`, lignes 76–155

6 Couche applicative STGA

Le payload STGS reste opaque au protocole externe. S'il commence par le magic STGA, il peut être interprété comme :

- **TEXT_MESSAGE** : séquence applicative + octets de texte ;
- **SIGNAL_BLOCK** : fréquence d'échantillonnage, fréquence nominale, amplitude, index absolu et échantillons float32.

Un payload sans magic STGA reste valide et est simplement considéré comme binaire historique.

Source : `include/stgs/ApplicationPayload.hpp`, lignes 1–150

7 Arrêt Ctrl+C sans handler C++ dangereux

Le processus bloque SIGINT/SIGTERM avant la création des workers. Un `std::jthread` dédié appelle `sigtimedwait()` toutes les 100 ms. Lorsqu'il reçoit le signal, il met l'atomic `running` à false. Les boucles réseau relisent cette valeur au plus tard après leur timeout de `poll()` (250 ms par défaut).

Pourquoi ne pas appeler directement du C++ depuis un signal handler ?

Les primitives C++ usuelles (mutex, iostream, allocations) ne sont pas garanties async-signal-safe. Le thread d'attente transforme le signal POSIX en événement ordinaire manipulable par le programme.

Source : `apps/ground_station/PosixSignalStopController.cpp`, lignes 21–138

8 Affichage terminal

`TerminalUi` active les couleurs uniquement sur un TTY, si `TERM` n'est pas `dumb`, si `NO_COLOR` n'est pas défini et si la CLI n'a pas demandé `-no-color`. Le texte reçu d'un autre processus est assaini : ESC et caractères de contrôle sont échappés afin qu'un message distant ne puisse pas injecter de séquence ANSI dans le terminal.

Source : `src/TerminalUi.cpp`, lignes 78–186

9 Garanties et non-garanties

Garanti dans cette maquette	Non garanti / volontairement hors périmètre
Validation taille, magic, version, CRC et champs de base	Chiffrement ou authentification réseau
Réassemblage TCP fragmenté et multi-clients	Identité d'un service détecté sur un port OPEN
Backpressure et ordre d'export déterministe	Temps réel dur / latence bornée au niveau OS
Replay STGF avec bornes avant allocation	Modélisation RF ou protocole spatial normatif
Signal synthétique + DSP lisible	Chaîne de traitement qualifiée ou certifiée